PulseChain: Technical Report

Predicting and Tracking HIV/ARV Supply Chain Delays Across East Africa

INEZA Bella Melissa, BSc Software Engineering, African Leadership University. Supervisor: KHWATENGE Immaculate Elvira. 29 October 2026.

Repository: [add GitHub link] Walkthrough video: [add video link]

1. Scope alignment and feature validation

PulseChain estimates how likely an HIV/ARV shipment is to arrive late, explains each estimate in plain language, and lets signed-in health workers report shortages and restocks. A dashboard combines country-level predicted risk with district-level reports. The table compares the approved proposal with the final product.

Planned feature	Intended purpose	Status	Evidence (repository path)
Clean SCMS data and leakage-safe features	Avoid training on future information	Completed	notebooks/PulseChain_Model_Improved.ipynb; app/models/history_snapshot.json
LR baseline vs LightGBM comparison	Show whether the complex model adds value	Completed (extended with a vendor-recency rule and time-series validation)	Notebook; Model page; app/models/initial_metrics.csv
SHAP reason codes	Make each score actionable	Completed (coefficients for LR)	app/services/risk_model.py (explain); Predict page
Shortage report and restock log	Capture what happens in the field	Completed	app/routers/reports.py; Report page
District-level dashboard	Combine predicted and reported risk	Completed, with a limitation: risk is country-level	app/services/analytics.py; Dashboard page
Role-based sign-in and data scoping	Users see only their district or country	Completed (added after supervisor feedback and ethics review)	app/auth.py; app/dependencies.py; tests/test_auth.py
Batch CSV scoring; delay-cause logging	Plan whole shipments; learn why delays happen	Completed (added)	app/routers/predictions.py, shipments.py; Batch, Shipments pages
Usability evaluation (SUS)	Judge ease of use	Partial: survey built; No participants have completed the SUS yet (n = 0), so no score is reported; this is a limitation and the next testing step	app/routers/feedback.py; Feedback page; data collection instrument
Deployment	Run on standard hardware	Partial: runs locally; Dockerfile written, cloud hosting not done	Dockerfile; README
SMS gateway; live national LMIS integration	Reach low-connectivity sites	Not implemented (excluded in the proposal scope)	Proposal section 1.5

Scope changes and justification. (1) The SCMS data record the destination country only, so prediction is country-level and a district inherits its country's risk; districts matter only through live reports. (2) Sign-in and roles were added so that people from one district or country cannot see data that does not belong to them, which the ethics application also relies on. (3) A simple vendor-recency rule was added as a baseline, following Müller et al. (2025), who found complex models did not beat simple baselines. (4) Streamlit was replaced by FastAPI and React so that the model, access rules and interface are separate, testable modules.

2. Technical implementation

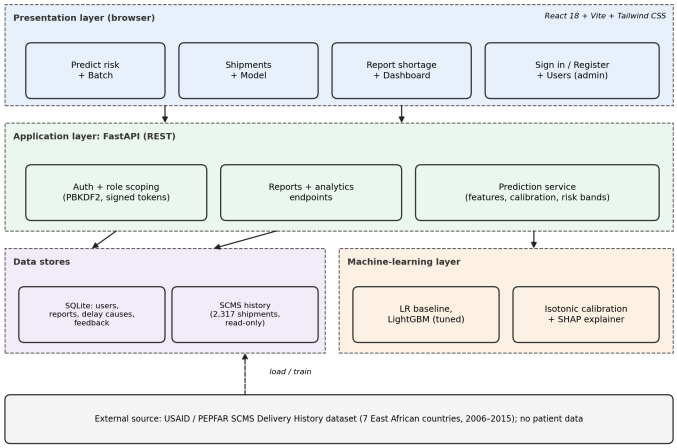


Figure 1. Architecture: React interface, FastAPI application layer, trained model, and data stores.

Frontend and backend interaction. The React 18 (Vite, Tailwind CSS, recharts) interface calls a REST API served by FastAPI. After sign-in, the browser sends a signed token with every request. The server, not the page, decides what each role may see: every reports, dashboard, analytics, shipments and prediction query is filtered by the user's district or country.

Data handling. Shipment data (2,317 shipments, 7 countries, 2006-2015, 12.3% late) are split chronologically: the oldest 80% for development and the newest 20% (464 shipments, 82 late) for testing. Reports, delay causes, users and survey answers are stored in SQLite with validated inputs (pydantic) and parameterised SQL.

Algorithms and custom logic. (a) Leakage-safe history features: a vendor's or country's recent late rate is computed only from shipments delivered at least 60 days before the current one; a count feature that acted as a clock (correlation 0.96) was removed. (b) Five-fold forward-chaining cross-validation with randomised search (30 settings) on average precision, plus feature-set selection by the one-standard-error rule. (c) Isotonic calibration applied with piecewise-linear interpolation, and displayed probabilities capped at 50%. (d) Percentile risk bands (High = top 15% of scores, Medium = next 25%). (e) SHAP tree explanations turned into plain sentences. (f) A district flag rule: at least two open shortages and country risk at least 1.15 times the overall level. (g) SUS scoring, PBKDF2-SHA256 password hashing, HMAC-signed expiring tokens, and a login rate limiter.

Model (held-out test set)	ROC-AUC	PR-AUC	Late found in top 20%	Lift
Rule: vendor recent late rate	0.731	0.356	40%	2.0x
Logistic Regression	0.734	0.402	46%	2.3x
LightGBM (default)	0.781	0.416	46%	2.3x
LightGBM (tuned), final model	0.768	0.344	39%	1.9x

The final model was chosen by cross-validation on development data only. Its ROC-AUC interval (about 0.72 to 0.81) overlaps the vendor rule, so the report claims no accuracy gain over the rule. The value lies in calibrated probabilities, explanations and a pipeline ready for real clinic data.

3. Code quality and object-oriented design

The backend began as a single 1,066-line script. It was refactored into 20 modules with one responsibility each, and the entry point (`main.py`) is now about 40 lines.

Layer	Modules	Responsibility
Entry and wiring	main.py, config.py, database.py	Create the app, shared constants, SQLite schema
Interface (routers)	routers/auth_routes, predictions, shipments, reports, dashboard, feedback, admin	HTTP endpoints only; no business logic
Business logic (services)	services/risk_model, explanations, analytics, users, demo_data, reports	Scoring, explanations, summaries, accounts
Cross-cutting	auth.py, dependencies.py, schemas.py	Passwords, tokens, role scope; sign-in checks; input models
Frontend	pages/*.jsx, components/ (Field, Select, Button, Charts), api.js	One page per screen; reusable form and chart components

OOP principles. *Encapsulation:* the `RiskModel` class owns the trained pipeline, calibration, thresholds and scored shipments, and exposes only `predict_proba, calibrate, band_for, explain, scored_shipments`; `LoginLimiter` hides its failure log. *Abstraction and separation of concerns:* routers call services and never touch the model or the database schema directly. *Modularity and reuse:* pure functions (explanations, scope checks) have no framework dependency, so they are unit tested alone; FastAPI dependency injection (`current_user, admin_only, manager_only`) reuses one access check across all routes. Inheritance and polymorphism are used lightly (pydantic models; the explainer switches between SHAP and coefficients depending on the model type). Naming is consistent, and the README documents the structure.

4. Testing, validation and results

Four strategies were used. Each was chosen to match a risk: wrong logic (unit), broken interactions or access leaks (integration), slow responses (performance) and poor usability (SUS).

Strategy	Why chosen	How conducted	Result and improvement
Unit tests	Check security and scoring logic in isolation	28 unittest cases in tests/: password hashing, token tampering and expiry, role scoping, login lock-out, calibration cap and floor, risk bands, explanation wording	28 of 28 pass. They run in under 1 second, so they can be repeated after every change.
Integration / functional (API)	Verify the layers work together, including errors and role limits	scripts/smoke_test.py runs 35 checks against the live server: sign-in, prediction, batch, shipments, reports, dashboard, analytics, demo tools, scoping	35 of 35 pass, before and after the module refactor. Testing the refactored code exposed a crash in the model-missing fallback, which was fixed.

Strategy	Why chosen	How conducted	Result and improvement
Performance and stress	NFR1 requires a prediction in under 2 seconds on a laptop	scripts/perf_test.py: 30 requests per endpoint, then 20 parallel users sending 200 predictions	See the table in section 5; zero errors in 200 requests.
Usability (SUS)	Judge ease of use with real users	3 tasks and the 10-item SUS, anonymous, per the ethics-approved instrument	No participants have completed the SUS yet (n = 0), so no score is reported; this is a limitation and the next testing step (benchmark 68).

Testing with different inputs. The system returns clear responses to invalid and unauthorised inputs:

Input	Expected behaviour	Observed
Wrong password; unapproved account	401; "waiting for approval" (403)	As expected
Request without sign-in	401 on every data endpoint	As expected
District officer asks for another country (predict, shipments, reports)	403 or filtered out	As expected
Non-admin calls demo tools or the user list	403	As expected
Unknown country, product or delay reason; survey with 3 answers	422 with a message	As expected
Restock a report twice; delete a missing report	400; 404	As expected
Model file missing	Labelled heuristic estimate, no crash	Works (unit-tested fallback)

5. Performance and compatibility

Environment tested: Apple Silicon Mac (arm64), macOS 26.1, Python 3.9.6, CPU only (no GPU). Latency was measured against the running server on the same machine.

Endpoint (30 requests each)	Mean	Median	p95	Max
GET /api/health	0.5 ms	0.5 ms	0.7 ms	0.7 ms
POST /api/predict (with SHAP explanation)	5.8 ms	4.7 ms	10.6 ms	18.1 ms
GET /api/shipments (ranked)	3.8 ms	3.4 ms	4.1 ms	9.3 ms
GET /api/dashboard	2.5 ms	2.4 ms	3.0 ms	3.4 ms
GET /api/analytics	2.6 ms	2.5 ms	3.1 ms	3.5 ms
Stress: 200 predictions, 20 parallel users	86.4 ms	81.0 ms	148.2 ms	219.7 ms

The stress run completed at 227 requests per second with no errors, and the slowest single prediction (6 ms) is far below the 2-second requirement. **Browsers and devices:** Google Chrome 156.0.8078.73 and Safari 26.1 on an Apple Silicon Mac (macOS); Predict, Dashboard and a narrow phone-width window were checked in both, with no errors. The interface uses responsive Tailwind layouts: the tab bar scrolls sideways on narrow screens and tables scroll inside their cards. Low-end hardware was not tested; the model is a small tree ensemble that needs no GPU.

6. Limitations and next steps

Results rest on 82 late test shipments from 2006-2015, so they are a proof of concept. The machine-learning models do not clearly beat the vendor rule. Districts inherit country risk because the data have no district field. Shortage reports in the demo are simulated and labelled. Next steps are a pilot with real facility reports, cloud deployment, password reset and email verification, and retraining on recent shipment data.